

# Malware Similarity Trends and Gaps Analysis

## Final Report

|               |                                                                            |
|---------------|----------------------------------------------------------------------------|
| Anmol Agarwal | <a href="mailto:Anmol.Agarwal@utdallas.edu">Anmol.Agarwal@utdallas.edu</a> |
| Gina Fossati  | <a href="mailto:Gina.Fossati@utdallas.edu">Gina.Fossati@utdallas.edu</a>   |
| Jennifer Ward | <a href="mailto:Jennifer.Ward@utdallas.edu">Jennifer.Ward@utdallas.edu</a> |

May 1, 2020

## Project Summary

### Description

People have been trying to address malware similarity both academically and operationally for decades, with no clear community consensus on techniques or even which malware features are most relevant. We need to look at the problem differently. Recent efforts at MITRE have proposed a new methodology focusing on the fingerprints left behind by the Malware Development Life Cycle (MDLC). Malware is continuously changing, and today's malware is characterized as yesterday's malware plus more. Since malware is software, it is important to understand its development life cycle. The life cycle can be broken down into eight phases: requirements, design, code, build, configure, package, collection, and analysis. We will focus on the first six phases because collection and analysis are not direct properties of the malware and its author. We will consider the process of a software developer in each of the phases and determine what we can extract from looking at the end product. Some additional things we will assess include the knowledge or skill level of the malware's author and links between phases of different malware that are similar due to unique choices in development and operation. In addition to the information found in each phase of the life cycle, we must also consider obfuscation in our analysis, such as heavily obfuscated namespaces, source code obfuscated by multiple layers, and control flow obfuscation where a single function could be broken into multiple functions and indirect calling. Our goal is to create a prototype that automates the analysis of features found in the phases of the Malware Development Life Cycle (MDLC) as seen in the final product or malware executable. From this analysis, we will attempt to identify similarities in malware.

## **Required Skills**

- Basic understanding of software engineering (e.g. waterfall, spiral, agile).
- Basic understanding of software development (e.g. how to compile a program).
- Comfortable reading programs written in a scripting language (e.g. Python, Powershell, PHP).
- Comfortable reading programs in a compiled language (e.g. C/C++, Delphi, Go, Rust).
- Comfortable reading disassembled x86 code.

## **Total Budget**

There were no monetary costs associated with this project. The team met approximately ten hours total per week, communicated daily, and held weekly status update meetings.

## **Team Responsibilities**

- Review the Malware Development Life Cycle (MDLC) and understand the phases.
  - Focus on adversary activities and how the malware is made (languages used, data structures, compiler linking options, etc.) to determine similarities in malware.
- Set up a malware analysis environment.
- Develop implementations of an exfiltration tool in various languages using different packaging and configuration options.
- Determine features found in each layer of the MDLC to extract from a malware executable.
- Develop a prototype of an automated tool to extract these features in order to compare similarities in malware.

## **Organization**

### **Plan**

The group met virtually once a week and communicated throughout the week through a messaging application to ensure progress on the project. The team used effective time management and communication to ensure that the project progressed on schedule.

## **Deliverables**

- Proposal & Literature Review
- Implementations of Exfiltration Tool
- Progress Report & Presentation
- Final Automated Tool Prototype
- Final Report & Presentation

## **Schedule**

February 7: Literature Review  
February 14: Proposal Due  
February 14: Understand phases of Malware Development Life Cycle and develop plan  
February 21: Set up Malware Analysis Environment  
February 21: Start Malware implementations  
February 28: Finish 1st Malware implementation  
March 6: Finish 2nd Malware implementation  
March 13: Finish 2nd Malware implementation  
Spring Break: Research packaging/obfuscating  
Brainstorm features from malware that indicate malware similarities  
Finalize components from MDLC that will be used in analysis  
April 3: Midterm Report Due  
April 10: Develop analysis tool  
April 17: Develop analysis tool  
April 24: Analyze different packagers/compilers using our tool  
Finish Final Report and Presentation Slides  
May 1: Presentation, Final Deliverables Due

## **Motivation**

As malware are addressed, it is important to have a clear community consensus on how to define and measure similarity. We are interested in becoming more familiar with malware features, analyzing malware, and detecting similarities and differences to classify malware “families”. All group members have experience with software engineering, methodologies, and development and are comfortable reading both scripting and compiled languages (e.g., Python, C/C++, Go). Additionally, most of our team members have completed coursework involving MIPS assembly, and some have experience with x86. We were excited to apply these skills to malware analysis and classification.

## **Broader Impact**

We developed a prototype of an automated technique to identify similar characteristics in malware based on attributes found in different phases of the Malware Development Life Cycle. This will provide insight into how malware is created and common adversarial techniques which can be used to detect the authors or operators of new malware and how it is different from previous malware.

## **Overview**

The main goal of this project is to develop a prototype of an automated tool that analyzes malware and outputs useful information about the malware. To achieve this, we researched the malware development life cycle and each wrote two implementations of an exfiltration tool in various languages based on provided requirements to provide a ground truth for our feature extraction tool. We compared the final product of our malware tools, in the form of Windows executables, to find similarities and differences evident from each phase of the MDLC. With this knowledge, we were able to develop our prototype to extract these features automatically from a piece of malware.

## **Team Approach and Justification**

We extracted features from the Windows executable versions of our implementations of data exfiltration tools in order to determine what components or “fingerprints” are detectable from the executable. We used our data exfiltration tools to test our malware feature extraction tool because it is easier to establish a ground truth with tools we developed ourselves than tools written by others. We developed six data exfiltration tools: two tools in Java, one tool in Go, and three tools in Python to have samples with language and author variability. We attempted to extract features from the executable that correspond to the phases of the Malware Development Life Cycle to allow us to analyze samples. We also packaged and compiled our data exfiltration tools in a variety of ways to see if packaging and compilation variations affect the features that our tool is able to extract.

## **Work Completed**

### **Malware Analysis Environment**

The team has been able to set-up and remotely access two computers to use as a malware analysis environment. One computer has a REMnux Virtual Machine installed that assisted the team with malware analysis.

## Exfiltration Tool Variations

The team has developed variations of an exfiltration tool that can extract files specified by the attacker from a victim's machine, obfuscate these files, and send them to an IP address or FQDN through an HTTP Post command. The tool comes with an additional deobfuscation tool or option so that once the adversary retrieves these files from the IP address or FQDN, the exfiltrated files are deobfuscated. The team has developed six exfiltration tools, in which each team member developed two implementations in two different languages. These implementations consist of three variations of an exfiltration tool in Python from three different authors, two variations written in Java from two different authors, and one exfiltration tool written in Go.

## Packaging and Executables

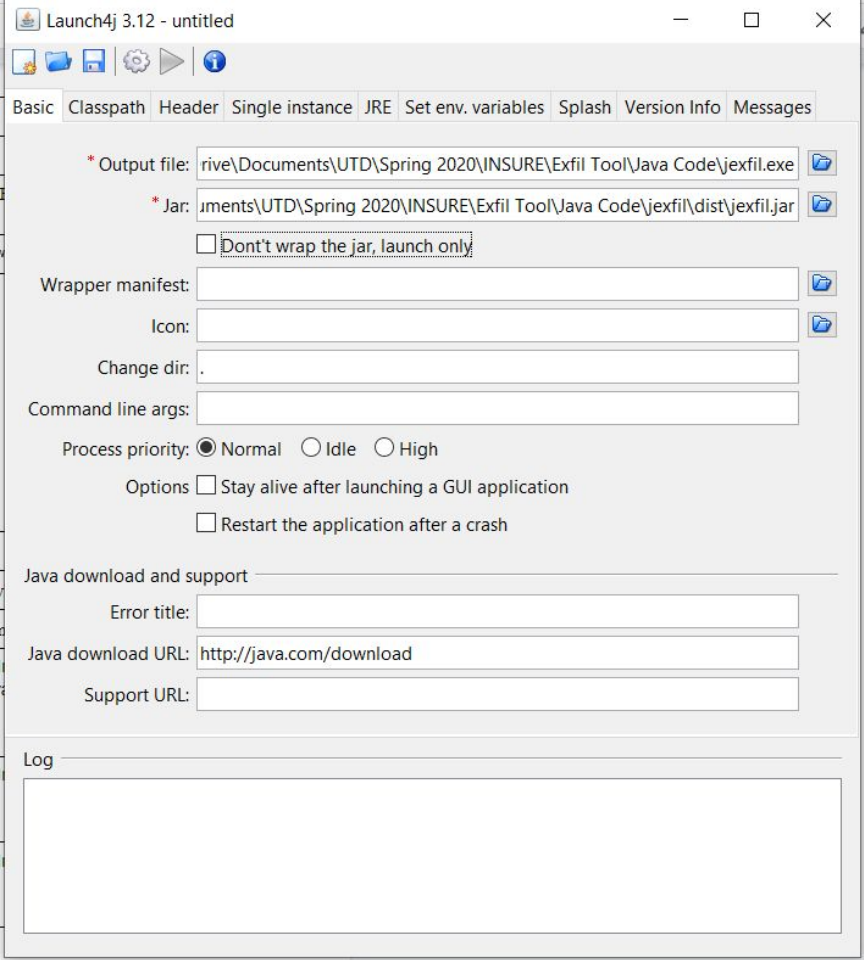
We applied obfuscation techniques to the exfiltration tools we developed by using various packaging tools and options. Specifically, we packaged all of the exfiltration tools as standalone Windows executables with various configuration options such that an attacker can directly run the executable on the victim's machine. This is important because malware analysts typically only work with the final product or self-contained executable version of the malware. The following tables outline the tools, options, and commands used for each programming language to convert the exfiltration tool implementations into Windows executable files.

| Language | Go               |                                       |                                |                                |
|----------|------------------|---------------------------------------|--------------------------------|--------------------------------|
| Tool     | go build package |                                       |                                |                                |
| Options  | Standard         | Switching off Optimization and inline | Disable DWARF debugging        | Switching off Debug Symbols    |
| Command  | go build tool.go | go build -gcflags="-N -l" tool.go     | go build -ldflags="-w" tool.go | go build -ldflags="-s" tool.go |

| Language | Python 3                                   |                                                                                           |                                                    |                                                            |
|----------|--------------------------------------------|-------------------------------------------------------------------------------------------|----------------------------------------------------|------------------------------------------------------------|
| Tool     | cx_freeze                                  | PyInstaller (The code was still written in Python 3, but PyInstaller ran from Python 2.7) |                                                    |                                                            |
| Options  | *setup.py                                  | Standard                                                                                  | Compiler Optimization                              | Encrypted                                                  |
| Command  | *create setup.py<br>python3 setup.py build | python -m PyInstaller --onefile<br>exfilTool.py                                           | python -O -m PyInstaller --onefile<br>exfilTool.py | python -m PyInstaller --key=1234 --onefile<br>exfilTool.py |

\*setup.py - configuration for cx\_freeze

```
from cx_Freeze import setup, Executable
setup(name = "exfilTool" ,
      version = "0.1" ,
      description = "" ,
      executables = [Executable("exfilTool.py")])
```

|                      |                                                                                     |
|----------------------|-------------------------------------------------------------------------------------|
| Language             | Java                                                                                |
| Tool                 | Launch4J                                                                            |
| Options              | N/A                                                                                 |
| Execution Screenshot |  |

## Feature Extraction

Once we had packaged executables, we created a list of features from each stage of the MDLC to extract from our executables. We then developed an automated tool that extracts these features from Windows executable malware.

## Results

We developed a tool that automates feature extraction from Windows executable malware. The currently supported features are listed below.

### Searching strings

Programming language (Go, Java, Python)

Python-specific:

Padding (“PADDINGXX”) present

If exe created from pyinstaller, use  
pyinstxtractor to extract:

Pyinstaller version

Python version

Length of package

Number of files in pyinstaller  
archive

Possible entry points

Java-specific:

Classes in source code package

Identified by ending in .classPK

Imported packages/classes/functions

Identified by java/\*

Go-specific:

Function calls (from main):

Identified by main.\*

Imported libraries/packages/functions:

Identified by

vendor/golang.org/x/\*

Identified by go.itab.\*

### Pefile

Timestamp for when executable was created

Architecture PE is supposed to be run on

Interpreted as a 32-bit or 64-bit binary

Warnings while parsing PE to determine  
anomalies (i.e. obfuscation)

Imported DLLs and their corresponding  
imported functions

Packers that match PEiD signatures

### Peiframe

Antidebug - Any DLLs or API function calls  
specific to anti-debugging

Antivm - Any DLLs or API function calls  
specific to anti-virtual machines

Packers

Crypto

Files and file types in the strings

URLs in the strings

IP addresses in the strings

Fuzzing in the strings

Antivirus

Metadata:

Comments

Company Name

File Description

File Version

Internal Name

Legal Copyright

Legal Trademarks

Original Filename

Product Name

Product Version

### Python-magic

File type

## Analysis

As seen in the output below, our tool (PEFeatExt.py) is able to extract the features listed above. For example, it identifies that this executable, tool.exe, used PyInstaller version 2.1+ to wrap the version 3.7 python file in a Windows executable.

Below the output, we have included screenshots of the difference in output of PEFeatExt when run on all pairings of our three PyInstaller executables. The executables were very similar but had a few differences. The standard and optimized variations had identical features except for the package length which was different for all three variations. The encrypted variation was the most different when compared to the standard and optimized variation with a different number of CArchive files, a compressed file and email found in the standard and optimized output, and an image and two database files detected in the encrypted version. Some of the detected files seem unlikely to be true files used in the executable. For example, it appears to be a coincidence that “.so” ended up at the end of the line starting with “0P },Q/”. However, it is still good to search for these extensions because a malware author may have obfuscated or encrypted these files giving them gibberish names.

### Output of PEFeatExt.py analyzing tool.exe which was converted to an executable using PyInstaller with default options:

```
Filename: tool.exe
Programming Language: Python
Timestamp: 2020-01-05 12:15:27
PADDINGXX: False
Pyinstaller:
    Pyinstaller version: 2.1+
    Python version: 37
    Length of package: 7876903 bytes
    Found 82 files in CArchive
    Possible entry point: pyiboot01_bootstrap
    Possible entry point: pyi_rth_certifi
    Possible entry point: tool
File Type: PE32+ executable (console) x86-64, for MS Windows
Machine: 64-bit (x86-x64)
Magic: 64-bit binary
Metadata:
Crypto:
    CRC32_poly_Constant
    CRC32_table
Antidebug:
```

GetLastError  
IsDebuggerPresent  
IsProcessorFeaturePresent  
RaiseException  
TerminateProcess  
UnhandledExceptionFilter

Antivm:

Packers:

Microsoft\_Visual\_Cpp\_80\_DLL

Files:

|                                                |            |
|------------------------------------------------|------------|
| %s%s.exe                                       | Executable |
| ucrtbase.dll                                   | Library    |
| base_library.zip                               | Compressed |
| KERNEL32.dll                                   | Library    |
| ADVAPI32.dll                                   | Library    |
| WS2_32.dll                                     | Library    |
| 'LI.gZ                                         | Compressed |
| 0P },Q/.so                                     | Executable |
| 44.gZ                                          | Compressed |
| =p.msg                                         | Email      |
| bVCRUNTIME140.dll                              | Library    |
| bapi-ms-win-core-console-l1-1-0.dll            | Library    |
| bapi-ms-win-core-datetime-l1-1-0.dll           | Library    |
| bapi-ms-win-core-debug-l1-1-0.dll              | Library    |
| bapi-ms-win-core-errorhandling-l1-1-0.dll      | Library    |
| bapi-ms-win-core-file-l1-1-0.dll               | Library    |
| bapi-ms-win-core-file-l1-2-0.dll               | Library    |
| bapi-ms-win-core-file-l2-1-0.dll               | Library    |
| bapi-ms-win-core-handle-l1-1-0.dll             | Library    |
| bapi-ms-win-core-heap-l1-1-0.dll               | Library    |
| bapi-ms-win-core-interlocked-l1-1-0.dll        | Library    |
| bapi-ms-win-core-libraryloader-l1-1-0.dll      | Library    |
| bapi-ms-win-core-localization-l1-2-0.dll       | Library    |
| bapi-ms-win-core-memory-l1-1-0.dll             | Library    |
| bapi-ms-win-core-namedpipe-l1-1-0.dll          | Library    |
| bapi-ms-win-core-processenvironment-l1-1-0.dll | Library    |
| bapi-ms-win-core-processthreads-l1-1-0.dll     | Library    |
| bapi-ms-win-core-processthreads-l1-1-1.dll     | Library    |
| bapi-ms-win-core-profile-l1-1-0.dll            | Library    |
| bapi-ms-win-core-rtlsupport-l1-1-0.dll         | Library    |
| bapi-ms-win-core-string-l1-1-0.dll             | Library    |
| bapi-ms-win-core-synch-l1-1-0.dll              | Library    |
| bapi-ms-win-core-synch-l1-2-0.dll              | Library    |
| bapi-ms-win-core-sysinfo-l1-1-0.dll            | Library    |
| bapi-ms-win-core-timezone-l1-1-0.dll           | Library    |

|                                                |            |
|------------------------------------------------|------------|
| bapi-ms-win-core-util-l1-1-0.dll               | Library    |
| bapi-ms-win-crt-conio-l1-1-0.dll               | Library    |
| bapi-ms-win-crt-convert-l1-1-0.dll             | Library    |
| bapi-ms-win-crt-environment-l1-1-0.dll         | Library    |
| bapi-ms-win-crt-filessystem-l1-1-0.dll         | Library    |
| bapi-ms-win-crt-heap-l1-1-0.dll                | Library    |
| bapi-ms-win-crt-locale-l1-1-0.dll              | Library    |
| bapi-ms-win-crt-math-l1-1-0.dll                | Library    |
| bapi-ms-win-crt-process-l1-1-0.dll             | Library    |
| bapi-ms-win-crt-runtime-l1-1-0.dll             | Library    |
| bapi-ms-win-crt-stdio-l1-1-0.dll               | Library    |
| bapi-ms-win-crt-string-l1-1-0.dll              | Library    |
| bapi-ms-win-crt-time-l1-1-0.dll                | Library    |
| bapi-ms-win-crt-utility-l1-1-0.dll             | Library    |
| blibcrypto-1_1-x64.dll                         | Library    |
| blibssl-1_1-x64.dll                            | Library    |
| bpython37.dll                                  | Library    |
| bpywintypes37.dll                              | Library    |
| bucrtbase.dll                                  | Library    |
| xbase_library.zip                              | Compressed |
| xcryptography-2.7-py3.7.egg-info\top_level.txt | Text       |
| %python37.dll                                  | Library    |

URLs:

IPs:

137.0.4.2

6.1.5.3

Fuzzing:

Antivirus:

Imports:

Library: KERNEL32.dll

Functions imported from KERNEL32.dll:

GetModuleFileNameW  
 GetProcAddress  
 GetCommandLineW  
 GetEnvironmentVariableW  
 SetEnvironmentVariableW  
 ExpandEnvironmentStringsW  
 GetTempPathW  
 WaitForSingleObject  
 Sleep  
 SetDllDirectoryW  
 CreateProcessW  
 GetStartupInfoW  
 LoadLibraryExW  
 CreateDirectoryW

GetShortPathNameW  
FormatMessageW  
LoadLibraryA  
MultiByteToWideChar  
WideCharToMultiByte  
GetExitCodeProcess  
GetLastError  
SetEndOfFile  
HeapReAlloc  
RtlCaptureContext  
RtlLookupFunctionEntry  
RtlVirtualUnwind  
UnhandledExceptionFilter  
SetUnhandledExceptionFilter  
GetCurrentProcess  
TerminateProcess  
IsProcessorFeaturePresent  
QueryPerformanceCounter  
GetCurrentProcessId  
GetCurrentThreadId  
GetSystemTimeAsFileTime  
InitializeSListHead  
IsDebuggerPresent  
GetModuleHandleW  
RtlUnwindEx  
SetLastError  
EnterCriticalSection  
LeaveCriticalSection  
DeleteCriticalSection  
InitializeCriticalSectionAndSpinCount  
TlsAlloc  
TlsGetValue  
TlsSetValue  
TlsFree  
FreeLibrary  
GetCommandLineA  
ReadFile  
CreateFileW  
GetDriveTypeW  
GetFileType  
CloseHandle  
PeekNamedPipe  
SystemTimeToTzSpecificLocalTime  
FileTimeToSystemTime  
GetFullPathNameW

GetFullPathNameA  
RemoveDirectoryW  
FindClose  
FindFirstFileExW  
FindNextFileW  
SetStdHandle  
SetConsoleCtrlHandler  
DeleteFileW  
GetStdHandle  
WriteFile  
ExitProcess  
GetModuleHandleExW  
GetACP  
HeapFree  
HeapAlloc  
GetConsoleMode  
ReadConsoleW  
SetFilePointerEx  
GetConsoleCP  
CompareStringW  
LCMapStringW  
GetCurrentDirectoryW  
FlushFileBuffers  
SetEnvironmentVariableA  
GetFileAttributesExW  
IsValidCodePage  
GetOEMCP  
GetCPInfo  
GetEnvironmentStringsW  
FreeEnvironmentStringsW  
GetStringTypeW  
GetProcessHeap  
WriteConsoleW  
GetTimeZoneInformation  
HeapSize  
RaiseException

Library: ADVAPI32.dll

Functions imported from ADVAPI32.dll:

ConvertStringSecurityDescriptorToSecurityDescriptorW

Library: WS2\_32.dll

Functions imported from WS2\_32.dll:

ntohl

PEiD Signature Matches:

Warnings:

diff standardPythonOutput encryptedPythonOutput

```
8,9c8,9
<      Length of package: 7876903 bytes
<      Found 82 files in CArchive
---
>      Length of package: 7907538 bytes
>      Found 84 files in CArchive
39,40c39,41
<      44.gZ
<      =p.msg
---
>      qe-KW=.DB
>      g\.Bmp
>      H`@3?+vX1j KQB^9RlRo.dB
```

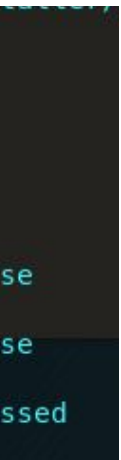


diff standardPythonOutput optimizedPythonOutput

```
8c8
<      Length of package: 7876903 bytes
---
>      Length of package: 7876941 bytes
```

diff encryptedPythonOutput optimizedPythonOutput

```
8,9c8,9
<      Length of package: 7907538 bytes
<      Found 84 files in CArchive
---
>      Length of package: 7876941 bytes
>      Found 82 files in CArchive
39,41c39,40
<      qe-KW=.DB
<      g\.Bmp
<      H`@3?+vX1j KQB^9RlRo.dB
---
>      44.gZ
>      =p.msg
```



## **Challenges**

It is difficult to detect common features among malware when the malware is obfuscated by its author or authors. An author or group of authors of malware may purposefully make variations in implementation language, configuration options, and other features when writing each new piece of malware to prevent this type of analysis from proving effective. Furthermore, some key “fingerprints” found in a certain phase of the MDLC may not be retrievable in the resulting executable that will be analyzed. It is very difficult to find wild malware samples that you have a ground truth for maliciousness since the source code is not provided. Therefore, we created our own data exfiltration tools so we know the objective, language, versions, libraries, functions, etc. that were used and can determine the accuracy of our feature extraction tool. This leads to another challenge - because our test set is so small, our tool may be biased towards the few executables we have used to determine features.

## **Future Work**

The analysis provided by our automated tool provides the foundation for future work in additional feature extraction and identifying similarities in malware, specifically features common to an author or group. It would also be fitting to test our tool on additional samples, especially real-world malware. In the future, the feature extraction tool could be used to help determine whether a given sample is malware or benign.

## **Literature Review**

### **1 Comparative Analysis of Feature Extraction Methods of Malware Detection**

This paper analyzes the general framework of malware detection systems and discusses the strengths and weaknesses of each method. Malware, including new, unfamiliar variants, tend to exhibit similar patterns in structure, which allows malware detection systems to detect unfamiliar malware through feature extraction. Current methods are based on two complementary approaches: signature-based detection and heuristic based approaches. The signature-based approach relies on the identification of unique string patterns in the binary code through a static technique, but it cannot detect zero-day attacks and attackers can easily work around this detection system through obfuscation. The heuristic approach is based on rules determined by experts by dynamically analyzing behavior to see when it deviates from normal behavior which effectively detects unfamiliar malware but tends to raise more false alarms and requires more resources and time. Others use a hybrid approach which integrates static and dynamic analysis

through feature extraction. Feature extraction is performed to reduce a large input or dataset into an organized, more manageable subset of information that requires less processing and computational overhead. Therefore, the feature extraction method used affects the performance of the system through efficiency, robustness, and accuracy. They conclude the paper with a performance overview of malware detection systems based on features. The efficiency and robustness of the system is defined by high accuracy, high True Positive Rate, and low False Positive Rate.

## **2 Malware Images: Visualization and Automatic Classification**

This paper describes a simple technique used to visualize and classify malware using image processing techniques. When malware is visualized as gray-scale images, those belonging to the same family tend to exhibit a similar layout and texture. This is most likely a result of the reuse of old malware binaries to create new variants. Due to this visual similarity, classification can be accomplished using standard image features rather than using code disassembly or execution as done in static and dynamic code analysis. This approach uses visual texture to automate classification by recognizing repeated patterns in the images through texture feature extraction. A common approach to feature extraction is accomplished through analyzing the frequency content of a texture block by dividing the frequency domain into rings and wedges such that features are found for those areas. They used GIST to compute texture features, which uses a wavelet decomposition of an image.

## **3 Novel Feature Extraction, Selection and Fusion for Effective Malware Family Classification**

In this paper, a machine learning based system is proposed that assigns malware to their appropriate malware families. A Microsoft malware dataset released to Kaggle was used for training and testing. In this approach, malware features are extracted from hex and assembly views. Hex dumps are used to extract metadata and entropy. The malware files are also disassembled to determine features such as frequencies of the following symbols: -, +, \*, ], [, ?, @, Operation Code, Register, and API. If the frequencies of the symbols are high, then that is indicative that the malware file is trying to evade detection. For classification, a Random Forest Algorithm was used to determine the importance of each feature category based on the mean decrease impurity. Each feature was ranked based on their Gini Impurity Scores. The feature that contributed most to the impurity was the metadata (size of the file, address of first bytes technique). The XGBoost algorithm was used with bagging and feature optimization to determine the feature categories and accuracy and LogLoss for the training and testing sets. With XGBoost, in the Hex Dump file, the features that had the highest 5CV accuracy were ENT (entropy) and 1G (1-gram features that represent byte frequency) with a 0.9862 and 0.9808

accuracy respectively. In the disassembled file the features that had the highest CV-5 accuracy were MISC (95 manually chosen keywords that could be extracted from the interpretation from IDA from the code and dll which shows the number of imported DLLs) and OPC (operation codes) with a 0.9917 accuracy and a 0.9907 accuracy respectively.

#### **4 Differentiating Malware from Cleanware Using Behavioural Analysis**

In this paper, malware features were extracted by examining the behavior of API calls in the malware files. Files are executed one by one and then execution logs are collected for each file. Runtime behavior was recorded for the binaries of the malware and cleanware. Then, trace reports using a tool HookMe were generated; these trace reports log various events, such as what API calls the binary file makes. A list of strings and string frequencies for each trace report is created, and then feature vectors are created using the original file's string frequencies. Some examples of string features extracted from trace reports are "Kernel32.dll" and "DontUseDNSLoadBalancing".

Once the string features were extracted, files were classified using the Weka library. Malware was classified using the built in Weka classifiers for classification such as Support Vector Machines (SVM), Random Forest, Instance- Based Classifier IB1, and Decision Table. Each file is also tested with k-fold cross validation. In this approach, malware and cleanware were classified and malware was differentiated from cleanware with a 97.4% accuracy.

#### **5 A Lifecycle Based Approach for Malware Analysis**

This paper uses the twelve stage lifecycle of behavioral activity to identify malware based on its behavior. This technique does not use signature based detection like many other methods. The lifecycle method breaks down into twelve classes: inbound scan, inbound attack, spontaneous attack, client-side exploit, egg download, device infection, local reconnaissance, network surveillance, C&C communications, peer coordination, attack preparation, and malicious outbound propagation. This method has favorable false positive, false negative, and accuracy compared to other techniques.

### **Biographical Sketches of Investigators**

#### **Anmol Agarwal**

Anmol Agarwal holds a Bachelor's degree in Software Engineering with a specialization in information assurance and is currently pursuing her Master's degree in Computer Science with a specialization in information assurance. She is also an NSF CyberCorps Scholarship for Service recipient. She has experience in software development, project management, virtual machines, and information security. Anmol has experience with solving CTF (Capture the Flag) challenges, such as challenges in web security and forensics. She has completed coursework in

computer networks, digital forensics, information security, artificial intelligence, operating systems, and data and applications security. She is currently taking machine learning and cloud security. In her free time, Anmol is an active member of Women in CyberSecurity and the Computer Security Group.

## **Gina Fossati**

Gina Fossati graduated with her Bachelor's degree in Computer Science in May 2019, and she is currently pursuing her Master's degree in Computer Science in the Information Assurance Track. She is also an NSF CyberCorps Scholarship for Service recipient. She has completed coursework in information security, digital forensics, IOT security, language based security, operating systems, computer networks, and machine learning. She is currently taking data and application security, web programming languages, and advanced algorithms.

## **Jennifer Ward**

Jennifer Ward graduated with her Bachelor of Science in Computer Engineering in December 2019 and is now pursuing her Master of Science in Computer Science with a specialization in information assurance. She is also an NSF CyberCorps Scholarship for Service recipient. She has experience in analyzing data, software development, and practicing and examining security attacks and vulnerabilities. She has completed coursework in systems security and malicious code analysis, computer networks, computer architecture, operating systems, software engineering, Linux operating system security, and embedded microprocessor systems. She is also currently taking data and application security and network security.

## **References**

- [1] Ranveer, Smita, and Swapnaja Hiray. "Comparative Analysis of Feature Extraction Methods of Malware Detection." *International Journal of Computer Applications*, vol. 120, no. 5, June 2015, [pdfs.semanticscholar.org/8ec8/858a25ce2fb76388d40ad07878b0ec79f580.pdf](https://pdfs.semanticscholar.org/8ec8/858a25ce2fb76388d40ad07878b0ec79f580.pdf).
- [2] L. Nataraj, S. Karthikeyan, G. Jacob, and B. S. Manjunath. 2011. Malware images: visualization and automatic classification. In Proceedings of the 8th International Symposium on Visualization for Cyber Security (VizSec '11). Association for Computing Machinery, New York, NY, USA, Article 4, 1–7. DOI:<https://doi.org/10.1145/2016904.2016908>
- [3] M. Ahmadi, D. Ulyanov, S. Semenov, M. Trofimov, G. Giacinto. March 2016. Novel Feature Extraction, Selection and Fusion for Effective Malware Family Classification. In

CODASPY '16: Proceedings of the Sixth ACM Conference on Data and Application Security and Privacy. Association for Computing Machinery, New York, NY, USA, pages 183 - 194.

[4] R. Tian, L.M. Batten, R. Islam, and S.C. Versteeg, "Differentiating Malware from Cleanware Using Behavioural Analysis," in Proc. of the 5th IEEE Int. Conf on Malicious and Unwanted Software (Malware'10), Nancy, France, pp. 23–30, October 2010

[5] S. K. Pandey and B. M. Mehtre, "A Lifecycle Based Approach for Malware Analysis," 2014 Fourth International Conference on Communication Systems and Network Technologies, Bhopal, 2014, pp. 767-771.